SLIMIRON + SIMAC

Design & Architecture Documentation

Version 0.3.0 · Hobby Cryptographic Primitive

Algorithm	Slimiron AEAD (stream cipher + SIMAC MAC)
Key size	256-bit
Nonce	96-bit (misuse-resistant via SIV)
Tag	128-bit
Security	Birthday bound 2^{128} (256-bit sponge capacity)
Rounds	14 (stream) / 10 (SIMAC)
Counter limit	$2^{32} - 2$ blocks ≈ 256 GB per (key, synth_nonce)
Status	Hobby project — NOT for production use

1. Overview

Slimiron is a nonce-misuse-resistant Authenticated Encryption with Associated Data (AEAD) scheme built entirely from scratch. It combines two original primitives:

- **Slimiron stream cipher** — an ARX-based counter-mode stream cipher using the SlimMix permutation.
- **SIMAC** — a duplex-sponge MAC with a 256-bit capacity region for 128-bit collision resistance.

The design goal is educational: understand every component at the mathematical level, with no external dependencies (not even standard hash functions).

Design Philosophy

All initialization constants are derived self-referentially from the Slimiron permutation itself — verified by gen_constants.py using pure Python with no hashlib or external libraries.

2. SlimMix — ARX Quarter-Round

SlimMix is the fundamental building block of both the stream cipher and the MAC. It is an Add-Rotate-XOR (ARX) construction operating on four 32-bit words.

2.1 Quarter-Round Definition

Given four 32-bit words (a, b, c, d), one SlimMix application performs:

```
a ← a + b    (mod 2³²)
d ← d ⊕ a
d ← d ⋘15

c ← c + d    (mod 2³²)
b ← b ⊕ c
b ← b ⋘11

a ← a + b    (mod 2³²)
d ← d ⊕ a
d ← d ⋘9

c ← c + d    (mod 2³²)
b ← b ⊕ c
b ← b ⋘5
```

The rotation constants (15, 11, 9, 5) were selected by sweeping 142 candidates and measuring avalanche quality. They satisfy:

- All odd — guarantees invertibility (gcd(rot, 32) = 1 for all odd rotations)
- Non-repeating — no two consecutive rotations are equal
- Full 512-bit avalanche achieved by round 2 (DR2)

2.2 Full Permutation

The permutation operates on a 16-word (512-bit) state. Each double-round applies column rounds then diagonal rounds — identical to the ChaCha layout:

```
for i in 0..(rounds/2):
  // Column round
  SlimMix(s[ 0], s[ 4], s[ 8], s[12])
  SlimMix(s[ 1], s[ 5], s[ 9], s[13])
  SlimMix(s[ 2], s[ 6], s[10], s[14])
  SlimMix(s[ 3], s[ 7], s[11], s[15])
  // Diagonal round
  SlimMix(s[ 0], s[ 5], s[10], s[15])
  SlimMix(s[ 1], s[ 6], s[11], s[12])
  SlimMix(s[ 2], s[ 7], s[ 8], s[13])
  SlimMix(s[ 3], s[ 4], s[ 9], s[14])
```

Default: 10 rounds (5 double-rounds). The column round mixes within columns; the diagonal round mixes across columns, ensuring every output word depends on every input word.

3. Slimiron Stream Cipher

3.1 State Layout (v0.2.1+)

The 512-bit state is divided into a rate region (words 0–7) and a capacity region (words 8–15):

Words	Contents	Role
state[0]	CTR	Counter — incremented each block
state[1..3]	N0, N1, N2	96-bit nonce (3 × 32-bit LE words)
state[4..7]	IV_0..IV_3	128-bit initialization vector (derived constant)
state[8..15]	K0..K7	256-bit key — permanently in capacity, never overwritten

Security Note

Placing the full 256-bit key in state[8..15] (capacity region) means it is NEVER XORed with input/output data and survives all permutation applications intact. Earlier versions incorrectly placed only 128 bits of key, silently discarding K4..K7.

3.2 Initialization

```
state[0]  ← 0                    // counter starts at 0
state[1]  ← LE32(nonce[0..3])
state[2]  ← LE32(nonce[4..7])
state[3]  ← LE32(nonce[8..11])
state[4]  ← IV_0  // slimiron_bootstrap("slimiron-stream-v5")[0]
state[5]  ← IV_1
state[6]  ← IV_2
state[7]  ← IV_3
state[8]  ← LE32(key[0..3])
...
state[15] ← LE32(key[28..31])
```

3.3 Block Generation

Each call to slimiron_block() generates 64 bytes of keystream:

```
slimiron_block(ctx):
  assert ctx.state[0] <= MAX_COUNTER   // 2³² - 2
  x ← copy(ctx.state)                 // working copy
  x ← permute(x, 10)                  // apply 10-round permutation
  for i in 0..16:
    x[i] ← x[i] + ctx.state[i]        // add-back (prevents inversion)
    stream[i*4..(i+1)*4] ← LE32(x[i])
  ctx.state[0]++                       // increment counter
```

The add-back step (x[i] + state[i]) is critical: it makes inversion of the permutation computationally infeasible even if the output stream is known.

3.4 Counter Limit

The counter is capped at `SLIMIRON_MAX_COUNTER = 0xFFFFFFFE` ($2^{32} - 2$). This gives a maximum of ($2^{32} - 1$) × 64 bytes ≈ 256 GB per (key, nonce) pair. Exceeding this limit returns `-2` and aborts the operation.

4. SIMAC — Sponge-Based MAC

4.1 Sponge Parameters

Parameter	Value	Notes
State size	512 bits (16 × 32-bit words)	Full permutation state
Rate (r)	256 bits (8 words, 32 bytes)	state[0..7] — XORed with input
Capacity (c)	256 bits (8 words, 32 bytes)	state[8..15] — never touched by absorb

Birthday bound	2^{128}	c/2 = 128 bits
Rounds	10	Per permutation call

4.2 Initialization

SIMAC initialization seeds the capacity region with derived constants XOR'd with the nonce, then immediately permutes the full state to mix key material throughout:

```
simac_init(ctx, mac_key[32], nonce[12]):
  // Rate: load 256-bit mac_key
  state[0..7] ← LE32(mac_key[0..31])

  // Capacity: derived constants XOR nonce (for per-nonce uniqueness)
  state[8]  ← SIMAC_INIT_0 ⊕ LE32(nonce[0..3])
  state[9]  ← SIMAC_INIT_1 ⊕ LE32(nonce[4..7])
  state[10] ← SIMAC_INIT_2 ⊕ LE32(nonce[8..11])
  state[11] ← SIMAC_INIT_3
  state[12] ← SIMAC_INIT_4
  state[13] ← SIMAC_INIT_5
  state[14] ← SIMAC_INIT_6
  state[15] ← SIMAC_INIT_7

  permute(state, 10)   // mix everything
  buffer ← 0[32]
  pos ← 0
```

4.3 Absorb

Data is absorbed into the sponge rate region through an internal 32-byte buffer. When the buffer fills, it is compressed into the state:

```
simac_absorb(ctx, data, len):
  // Fill partial buffer first
  if ctx.pos > 0:
    want ← RATE_BYTES - ctx.pos
    if len < want: buffer(ctx.pos..ctx.pos+len) ← data; ctx.pos += len; return
    buffer(ctx.pos..) ← data[0..want]
    data += want; len -= want
    compress(ctx)   // flush buffer into state

  // Fast path: full blocks directly (no buffer copy)
  while len >= RATE_BYTES:
    for i in 0..8: state[i] ⊕= LE32(data[i*4..])
    permute(state, 10)
    data += 32; len -= 32

  // Remaining partial block into buffer
  if len > 0: buffer[0..len] ← data; ctx.pos ← len
```

The fast path bypasses the buffer for full blocks, making bulk absorption efficient. The byte-by-byte and bulk paths produce identical results (verified by test_simac_fast_path).

4.4 Padding

Two-marker padding scheme (same as used in Ascon and similar sponge constructions):

```
simac_pad(ctx):
  buffer[pos]           ⊕= 0x01   // start marker
  buffer[RATE_BYTES-1] ⊕= 0x80   // end marker
  // When pos == RATE_BYTES-1: both bits land on same byte → 0x81
  compress(ctx)
```

This padding is injective: no two distinct (message, length) pairs produce the same padded block, ensuring the MAC is domain-separated from other inputs.

4.5 Domain Separation

After each logical field, the caller pads explicitly, then calls simac_domain() to XOR a unique constant into the first capacity word (state[8]) and permute:

```
simac_domain(ctx, dom):
  state[8] ⊕= dom          // only touches capacity — unreachable by absorb path
  permute(state, 10)
```

Domain	Constant	Label
SIMAC_DOMAIN_AAD	0x8439c00f	simac-domain-aad-v5
SIMAC_DOMAIN_CT	0x35ef9605	simac-domain-ct-v5
SIMAC_DOMAIN_SIV	0x493ccf67	simac-domain-siv-v5

Domain Separation Correctness

XORing into state[8] (first capacity word) is safe because the absorb path NEVER touches state[8..15]. The domain constant is therefore structurally separate from all absorbed data, preventing any ambiguity between AAD, ciphertext, and SIV inputs.

4.6 Finalization

Finalization applies a final padding, XORs a 256-bit mask into the full capacity region, then runs two extra permutations before squeezing the 128-bit tag:

```
simac_finalize(ctx, tag[16]):
  simac_pad(ctx)             // final padding + compress
  state[8]  ⊕= SIMAC_FINAL_0
  state[9]  ⊕= SIMAC_FINAL_1
  ...
  state[15] ⊕= SIMAC_FINAL_7
  permute(state, 10)         // first squeezing permutation
  permute(state, 10)         // second squeezing permutation
  tag ← LE32(state[0]) || LE32(state[1]) || LE32(state[2]) || LE32(state[3])
```

The double permutation in finalization provides additional margin: even if one permutation were partially weakened, the second provides a second layer before output is extracted.

5. SIV — Synthetic IV (Nonce-Misuse Resistance)

Standard stream ciphers catastrophically fail when a nonce is reused: two ciphertexts encrypted under the same (key, nonce) pair reveal their XOR, leaking plaintext. Slimiron addresses this with a Synthetic IV (SIV) construction.

5.1 SIV Derivation

```
siv_derive(key, nonce, aad, msg) → synth_nonce[12]:
  // Derive a dedicated SIV key from the stream cipher
  c ← slimiron_init(key, nonce)
  siv_key ← slimiron_derive_mac_key(c)    // first 32 bytes of keystream

  // Run SIMAC over: original nonce || aad || msg
  m ← simac_init(siv_key, nonce)
  simac_absorb(m, nonce, 12)              // commit original nonce
  simac_absorb(m, aad, aad_len)
  simac_absorb_len(m, aad_len)            // length field, big-endian
  simac_absorb(m, msg, mlen)
  simac_absorb_len(m, mlen)
  simac_pad(m)
  simac_domain(m, SIMAC_DOMAIN_SIV)
  tag ← simac_finalize(m)

  synth_nonce ← tag[0..12]               // truncate to 96 bits
```

5.2 SIV Security Properties

Scenario	Result	Security
Same key+nonce, same msg+aad	Same synth_nonce	Deterministic (expected)
Same key+nonce, different msg	Different synth_nonce	Nonce reuse harmless
Different key	Different synth_nonce	Key-binding
Tampered synth_nonce	Auth failure (-1)	Integrity protected

The key insight: the actual encryption uses synth_nonce (derived from the message content) as the nonce, not the user-provided nonce. Even if a user reuses the same nonce for two different messages, the synth_nonces will differ, preventing keystream reuse.

6. Full AEAD Construction

6.1 Encrypt

```
slimiron_aead_encrypt(key, nonce, aad, msg) → (synth_nonce, ciphertext, tag):

  // Step 1: SIV — derive synthetic nonce from (key, nonce, aad, msg)
  synth_nonce ← siv_derive(key, nonce, aad, msg)

  // Step 2: Derive 256-bit MAC key from keystream block 0
  c ← slimiron_init(key, synth_nonce)
  mac_key ← slimiron_derive_mac_key(c)  // keystream bytes 0..31
  // counter is now 1

  // Step 3: Initialize SIMAC over (mac_key, synth_nonce)
  m ← simac_init(mac_key, synth_nonce)

  // Step 4: Absorb AAD with domain separation
  simac_absorb(m, aad, aad_len)
  simac_absorb_len(m, aad_len)      // big-endian 64-bit length
  simac_pad(m)
  simac_domain(m, SIMAC_DOMAIN_AAD)

  // Step 5: Encrypt message + absorb ciphertext into SIMAC
  for each 64-byte block i of msg:
    keystream_block ← slimiron_block(c)   // counter = i+1
    ciphertext[i] ← msg[i] ⊕ keystream_block
    simac_absorb(m, ciphertext[i], 64)

  // Step 6: Finalize tag
  simac_absorb_len(m, mlen)         // ciphertext length, big-endian
  simac_pad(m)
  simac_domain(m, SIMAC_DOMAIN_CT)
  tag ← simac_finalize(m)

  // Zeroize all key material
  slim_zero(mac_key, 32)
  slim_zero(c, sizeof(c))
  slim_zero(m, sizeof(m))
```

6.2 Decrypt

```
slimiron_aead_decrypt(key, synth_nonce, ciphertext, tag, aad, wire_version) →
plaintext | -1 | -4:

  // Step 0: Reject mismatched wire version   if wire_version !=
SLIMIRON_WIRE_VERSION: return -4    // Step 1: Derive MAC key (same as encrypt)
  c ← slimiron_init(key, synth_nonce)
  mac_key ← slimiron_derive_mac_key(c)

  // Step 2: Recompute tag over (aad, ciphertext)
  m ← simac_init(mac_key, synth_nonce)
  simac_absorb(m, aad, aad_len)
  simac_absorb_len(m, aad_len)
  simac_pad(m)
  simac_domain(m, SIMAC_DOMAIN_AAD)
  simac_absorb(m, ciphertext, clen)
  simac_absorb_len(m, clen)
  simac_pad(m)
  simac_domain(m, SIMAC_DOMAIN_CT)
  calc_tag ← simac_finalize(m)

  // Step 3: Verify tag (constant-time)
  if crypto_verify_16(calc_tag, tag) != 0:
    slim_zero(output, clen)   // always zero output on failure
    return -1

  // Step 4: Decrypt only after verification
  for each 64-byte block i of ciphertext:
    keystream_block ← slimiron_block(c)
    plaintext[i] ← ciphertext[i] ⊕ keystream_block
```

Verify-First Decryption

Plaintext is NEVER released before the tag is verified. This prevents padding oracle and chosen-ciphertext attacks that exploit partial decryption output.

6.3 Wire Format

The on-wire message format is:

Field	Size	Description
version	1 byte	SLIMIRON_WIRE_VERSION (0x03); decrypt rejects mismatches
synth_nonce	12 bytes	Synthetic nonce (first 12 bytes of SIV tag)
ciphertext	mlen bytes	Encrypted message
tag	16 bytes	SIMAC authentication tag

Total overhead: 29 bytes per message (1 version + 12 + 16).

7. Self-Bootstrapped Constants

All initialization constants are derived from the Slimiron permutation itself — there is no dependency on SHA-256 or any external hash function.

7.1 Bootstrap Procedure

```
slimiron_bootstrap(label: ASCII string) → words[]:
  state ← [0] * 16
  buffer ← [0] * 32
  pos ← 0

  // Absorb label bytes into rate region
  for byte in label:
    buffer[pos] ⊕= byte
    pos++
    if pos == 32:                    // full block
      for i in 0..8: state[i] ⊕= LE32(buffer[i*4..])
      state ← permute(state, 10)
      buffer ← [0]*32; pos ← 0

  // Two-marker padding
  buffer[pos]    ⊕= 0x01
  buffer[31]     ⊕= 0x80
  for i in 0..8: state[i] ⊕= LE32(buffer[i*4..])
  state ← permute(state, 10)

  return state[0..N]                // squeeze N words
```

7.2 Derived Constants

Constant	Label	Value (hex)
SLIMIRON_IV_0	slimiron-stream-v5	0xb9e3ef7f
SLIMIRON_IV_1	slimiron-stream-v5	0x7638101d
SLIMIRON_IV_2	slimiron-stream-v5	0x53373520
SLIMIRON_IV_3	slimiron-stream-v5	0x654cbc86
SIMAC_INIT_0..7	simac-init-v5	8 words — see header
SIMAC_FINAL_0..7	simac-final-v5	8 words — see header
SIMAC_DOMAIN_AAD	simac-domain-aad-v5	0x8439c00f
SIMAC_DOMAIN_CT	simac-domain-ct-v5	0x35ef9605
SIMAC_DOMAIN_SIV	simac-domain-siv-v5	0x493ccf67

The "-v5" suffix in each label distinguishes these from SHA-256-derived constants used in earlier versions. The verifier script gen_constants.py recomputes all constants in pure Python and checks them against the hardcoded values in slimiron.h.

8. Security Analysis

8.1 Authentication Security

SIMAC's 256-bit capacity gives a birthday-bound collision resistance of 2^{128} MAC computations. An attacker attempting to forge a tag must either:

- Find a collision in the sponge capacity (requires ≈ 2^{128} operations), or
- Invert the SlimMix permutation (computationally infeasible with 10 rounds), or
- Mount a distinguishing attack on the keystream (requires breaking the stream cipher).

8.2 Constant-Time Comparison

```
crypto_verify_16(a[16], b[16]) → 0 | -1:
  volatile uint8_t diff ← 0
  for i in 0..16: diff |= a[i] ⊕ b[i]
  return (diff == 0) ? 0 : -1
```

The volatile accumulator prevents the compiler from short-circuiting the loop (early exit on first differing byte would leak timing information about the tag position).

8.3 Key Material Zeroization

```
slim_zero(ptr, len):
  volatile uint8_t *p ← ptr
  while len--: *p++ ← 0
  __asm__ volatile("" ::: "memory")   // compiler barrier
```

The compiler memory barrier prevents Link-Time Optimization (LTO) and Whole-Program Optimization (WPO) from treating the zeroing writes as dead stores. Without this barrier, an optimizing compiler may legally elide the writes since the buffer is never read afterwards.

8.4 Limitations

WARNING

Slimiron has NOT been cryptanalyzed by independent experts. The SlimMix permutation design is novel and may contain weaknesses not yet discovered. Do not use for any security-sensitive purpose. This is a learning project only.

- No formal security proof
- SlimMix rotation constants selected by empirical avalanche sweep, not differential/linear cryptanalysis

- 10 rounds may be insufficient — ChaCha uses 20 rounds with similar construction (stream rounds raised to 14 in v0.3.0)
- No side-channel analysis (cache timing, power analysis)

9. Performance

9.1 Benchmark Results

Operation	Throughput	Time/call (16 MB)
Encrypt	162 MB/s (AVX2)	~98 ms (14 rounds)
Decrypt	261 MB/s (AVX2)	~61 ms (14 rounds)

Conditions: GitHub Codespace (4-core, 16 GB RAM), AVX2 enabled, 14 stream rounds, 16 MB data, 50 iterations, 3 warmup rounds. ~25% overhead increase vs v0.2.2 due to rounds raised from 10 to 14.

Encrypt is slower than decrypt because it requires an extra SIMAC pass over the plaintext for SIV derivation. Decrypt skips this and verifies against the received synth_nonce directly.

9.2 Encryption vs Decryption Cost

Phase	Encrypt	Decrypt
SIV derivation (SIMAC over plaintext)	Yes (+1 pass)	No
MAC key derivation (1 keystream block)	Yes	Yes
SIMAC over AAD	Yes	Yes
Encrypt/Decrypt blocks (XOR + counter)	Yes	Yes
SIMAC over ciphertext	Yes	Yes
Tag verification	No	Yes

9.3 XOR Path

Two XOR implementations are compiled conditionally:

- **AVX2**: Two 256-bit SIMD loads, XOR, stores — 64 bytes in ~2 instructions
- **Scalar (fallback)**: 8× 64-bit integer XOR — unrolled for good pipelining

10. Test Suite

Test	What it verifies
test_vector	Deterministic output for fixed (key, nonce, msg, aad)
test_roundtrip	Encrypt→Decrypt recovers original plaintext
test_multiblock	Correct behavior at block boundary sizes (0,1,31,32,33...513)
test_avalanche	1-bit plaintext change flips ≥40 of 128 tag bits
test_siv_misuse	Nonce reuse with different msg → different synth_nonce
test_forgery	Cipher/tag/synth_nonce tamper → returns -1, output zeroed
test_aad	Tampered AAD detected by tag verification
test_domain_sep	AAD and ciphertext bytes are not interchangeable in MAC
test_wrong_credentials	Wrong key → auth failure; wrong nonce → uses synth_nonce correctly
test_empty	Zero-length message and AAD handled correctly
test_counter_overflow	slimiron_block() returns -2 when counter > MAX_COUNTER
test_slim_zero	slim_zero() fully clears buffer
test_verify16	crypto_verify_16() returns 0/−1 correctly
test_simac_fast_path	Bulk absorb == byte-by-byte absorb (identical tags)
test_capacity_isolation	Corrupted capacity word → different tag
test_xor64	xor64() matches reference byte-by-byte XOR
test_collision	10,000 random messages produce 0 tag collisions
`test_alias_guard`	encrypt returns -3 when msg == cipher (in-place aliasing UB guard)
`test_wire_version`	decrypt returns -4 on version mismatch; correct version succeeds
`test_round_count`	SLIMIRON_ROUNDS == 14; SIMAC_ROUNDS == 10 (compile-time assertion)

11. Changelog

Version	Fix	Type	Description
v0.2.1	#1	PERF	simac_domain() no longer calls simac_pad() internally — saves 2 permutations on both encrypt and decrypt paths
v0.2.1	#2	SECURITY	slim_zero() compiler barrier prevents LTO/WPO dead-store

			elimination of key material
v0.2.1	#3	SECURITY	Full 256-bit key placed in capacity region (state[8..15]); old layout silently discarded K4..K7 — BREAKING
v0.2.1	#4	COMPAT	simac_absorb_len() now encodes length big-endian (MSB first), matching TLS/HKDF/GCM convention — BREAKING
v0.2.2	#5	PURITY	All constants derived from Slimiron permutation itself; SHA-256 dependency removed — BREAKING
v0.3.0	#6	SECURITY	Stream rounds raised 10 → 14 (capacity argument applies to sponge, not stream cipher) — BREAKING
v0.3.0	#7	COMPAT	Wire format now includes 1-byte SLIMIRON_WIRE_VERSION (0x03); slimiron_aead_decrypt gains wire_version param, returns -4 on mismatch — BREAKING
v0.3.0	#8	CORRECTNESS	slimiron_aead_encrypt returns -3 if msg == cipher (aliasing UB guard)
v0.3.0	#9	SECURITY	crypto_verify_16 hardened with SLIM_COMPILER_BARRIER() after accumulation loop
v0.3.0	#10	CORRECTNESS	Removed pos field from slimiron_ctx (set inconsistently, never used for streaming)
v0.3.0	#11	CLARITY	simac_finalize contract documented: performs its own internal simac_pad(); callers must not pad before calling it
v0.3.0	#12	CORRECTNESS	Removed inconsistent if (aad_len > 0) guards; simac_absorb(len=0) is canonical no-op